

Aim: Create a system that uses blockchain to verify the authenticity of documents.

1. Create a file named document_verifier.py

```
import hashlib
import json
import time
class Block:
    """Represents a single immutable entry in our document ledger."""
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash
        }, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

class DocumentBlockchain:
    """Manages the registration and decentralized verification of documents."""
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, time.time(), "Genesis Block - Document Verification System", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_document(self, doc_name, doc_content):
        doc_hash = hashlib.sha256(doc_content.encode()).hexdigest()

        block_data = {
            "document_name": doc_name,
            "document_hash": doc_hash,
            "status": "Verified"
        }
        new_block = Block(
            index=len(self.chain),
            timestamp=time.time(),
            data=block_data,
            previous_hash=self.get_latest_block().hash
        )
        self.chain.append(new_block)
        return doc_hash
```

```
def verify_document(self, doc_content):
    target_hash = hashlib.sha256(doc_content.encode()).hexdigest()

    for block in self.chain[1:]: # Skip genesis block
        if block.data.get("document_hash") == target_hash:
            return True, block.data, block.timestamp
    return False, None, None

def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i-1]

        if current.hash != current.calculate_hash():
            return False
        if current.previous_hash != previous.hash:
            return False
    return True

if __name__ == "__main__":
    sys_blockchain = DocumentBlockchain()
    print("--- [1] Initializing Blockchain Document Verification System ---")

    invoice_content = "Invoice #10042 - Total Due: $1,500.00 - Vendor: Acme Corp"
    degree_content = "University Diploma - Student: Jane Doe - Major: Computer Science"

    inv_hash = sys_blockchain.add_document("Acme_Invoice.txt", invoice_content)
    deg_hash = sys_blockchain.add_document("Jane_Doe_Diploma.txt", degree_content)

    print(f'Registered 'Acme_Invoice.txt' | Hash: {inv_hash}')
    print(f'Registered 'Jane_Doe_Diploma.txt' | Hash: {deg_hash}\n')

    print("--- [2] Verifying Untampered Original Document ---")
    test_doc_1 = "Invoice #10042 - Total Due: $1,500.00 - Vendor: Acme Corp"
    is_valid, meta, ts = sys_blockchain.verify_document(test_doc_1)
    print(f'Verification Result: {is_valid}')
    if is_valid:
        print(f'MetaData Match: '{meta['document_name']}' matched on ledger.\n')

    print("--- [3] Verifying Tampered/Fraudulent Document ---")
    fake_doc_1 = "Invoice #10042 - Total Due: $15,000.00 - Vendor: Acme Corp"
    is_valid, _, _ = sys_blockchain.verify_document(fake_doc_1)
    print(f'Verification Result: {is_valid}')
    print(f'Reason: System rejects it because the modified string generates an unlisted hash.\n')

    print("--- [4] Auditing Blockchain Integrity ---")
    print(f'Is blockchain ledger structurally secure and untampered? {sys_blockchain.is_chain_valid()}')
```

```
C:\BCTPRAC>python document_verifier.py
--- [1] Initializing Blockchain Document Verification System ---
Registered 'Acme_Invoice.txt' | Hash: aea71b2ee3220677ff3bbdf44f4fffc60ff20f604d4bd25cfcc8cf2274ffa0af0
Registered 'Jane_Doe_Diploma.txt' | Hash: a13248c849cc5721679af67de68a9cccd35edc06c22fa7c618dbaaa69a19ceba\n
--- [2] Verifying Untampered Original Document ---
Verification Result: True
MetaData Match: 'Acme_Invoice.txt' matched on ledger.\n
--- [3] Verifying Tampered/Fraudulent Document ---
Verification Result: False
Reason: System rejects it because the modified string generates an unlisted hash.\n
--- [4] Auditing Blockchain Integrity ---
Is blockchain ledger structurally secure and untampered? True
```

2. Once the file is verified by static input, it's time for user input

Create a new file named `document_verifier_input.py`

```
import hashlib
```

```
import json
```

```
import time
```

```
class Block:
```

```
    def __init__(self, index, timestamp, data, previous_hash):
```

```
        self.index = index
```

```
        self.timestamp = timestamp
```

```
        self.data = data
```

```
        self.previous_hash = previous_hash
```

```
        self.hash = self.calculate_hash()
```

```
    def calculate_hash(self):
```

```
        block_string = json.dumps({
```

```
            "index": self.index,
```

```
            "timestamp": self.timestamp,
```

```
            "data": self.data,
```

```
            "previous_hash": self.previous_hash
```

```
        }, sort_keys=True).encode()
```

```
        return hashlib.sha256(block_string).hexdigest()
```

```
class DocumentBlockchain:
```

```
    def __init__(self):
```

```
        self.chain = [self.create_genesis_block()]
```

```
    def create_genesis_block(self):
```

```
        return Block(0, time.time(), "Genesis Block - Document Verification System", "0")
```

```
    def get_latest_block(self):
```

```
        return self.chain[-1]
```

```
    def add_document(self, doc_name, doc_content):
```

```
        doc_hash = hashlib.sha256(doc_content.encode()).hexdigest()
```

```
        block_data = {
```

```
            "document_name": doc_name,
```

```
            "document_hash": doc_hash,
```

```
            "status": "Verified"
```

```
        }
```

```
        new_block = Block(
```

```
            index=len(self.chain),
```

```

        timestamp=time.time(),
        data=block_data,
        previous_hash=self.get_latest_block().hash
    )
    self.chain.append(new_block)
    return doc_hash
def verify_document(self, doc_content):
    target_hash = hashlib.sha256(doc_content.encode()).hexdigest()
    for block in self.chain[1:]:
        if block.data.get("document_hash") == target_hash:
            return True, block.data, block.timestamp
    return False, None, None
if __name__ == "__main__":
    sys_blockchain = DocumentBlockchain()

    original_invoice = "Invoice #10042 - Total Due: $1,500.00 - Vendor: Acme Corp"
    sys_blockchain.add_document("Acme_Invoice.txt", original_invoice)

    print("=====")
    print(" BLOCKCHAIN DOCUMENT VERIFIER (INTERACTIVE MODE) ")
    print("=====")
    print("System loaded with 1 registered document:")
    print(f"-> Content: \"{original_invoice}\"")
    print("Type 'exit' at any prompt to quit the program.\n")

    while True:
        print("Choose an option:")
        print("1. Register a NEW document to the blockchain")
        print("2. VERIFY an existing document string")
        choice = input("Enter choice (1 or 2): ").strip()

        if choice.lower() == 'exit':
            break

        if choice == "1":
            name = input("\nEnter a name for this document (e.g., contract.txt): ")
            content = input("Enter or paste the exact document text: ")

            doc_hash = sys_blockchain.add_document(name, content)
            print(f"SUCCESS: Document registered! Generated SHA-256
Fingerprint:\n{doc_hash}\n")
            print("-" * 50)

        elif choice == "2":
            content = input("\nEnter/paste the text of the document you want to verify: ")

            is_valid, meta, timestamp = sys_blockchain.verify_document(content)

            print("\n--- VERIFICATION RESULT ---")
            if is_valid:

```

```

print("STATUS: VALID AND UNTAMPERED!")
print(f"Document Name on Ledger: {meta['document_name']}")
print(f"Timestamp Logged      : {time.ctime(timestamp)}")
else:
    print("STATUS: INVALID / ALTERED!")
    print("Reason: This text does not match any registered hash in the blockchain
ledger.")
print("-" * 50 + "\n")
else:
    print("Invalid option. Please type 1 or 2.\n")

```

```

C:\BCTPRAC>python document_verifier_input.py
=====
BLOCKCHAIN DOCUMENT VERIFIER (INTERACTIVE MODE)
=====
System loaded with 1 registered document:
-> Content: "Invoice #10042 - Total Due: $1,500.00 - Vendor: Acme Corp"

Type 'exit' at any prompt to quit the program.

```

```

Choose an option:
1. Register a NEW document to the blockchain
2. VERIFY an existing document string
Enter choice (1 or 2): 1

Enter a name for this document (e.g., contract.txt): doc1
Enter or paste the exact document text: Invoice #10042 - Total Due: $19,500.00 - Vendor: Acme Corp
SUCCESS: Document registered! Generated SHA-256 Fingerprint:
30f2300764229653baeec3f7ecae65952285cb599097d6d22771dd33f9353053
-----

```

```

Choose an option:
1. Register a NEW document to the blockchain
2. VERIFY an existing document string
Enter choice (1 or 2): 2

Enter/paste the text of the document you want to verify: Invoice #10042 - Total Due: $1,500.00 - Vendor: Acme Corp

--- VERIFICATION RESULT ---
STATUS: VALID AND UNTAMPERED!
Document Name on Ledger: Acme_Invoice.txt
Timestamp Logged      : Fri Aug 14 23:54:33 2026
-----

```

```

Choose an option:
1. Register a NEW document to the blockchain
2. VERIFY an existing document string
Enter choice (1 or 2): 2

Enter/paste the text of the document you want to verify: Invoice #10042 - Total Due: $1,500.00 - Vendor: Suraj Comp

--- VERIFICATION RESULT ---
STATUS: INVALID / ALTERED!
Reason: This text does not match any registered hash in the blockchain ledger.
-----

```

```

Choose an option:
1. Register a NEW document to the blockchain
2. VERIFY an existing document string
Enter choice (1 or 2): 4
Invalid option. Please type 1 or 2.

```

```

Choose an option:
1. Register a NEW document to the blockchain
2. VERIFY an existing document string
Enter choice (1 or 2): exit

```

```
C:\BCTPRAC>
```

3. After verifying a document, next we will verify a file with some input
Create a text document with some content written in it and save it in any folder (best if saved where other python files are saved)
Once the txt file is created, create a new document_verifier_file.py file

```
import hashlib
import json
import time
import os

class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash
        }, sort_keys=True).encode()
        return hashlib.sha256(block_string).hexdigest()

class DocumentBlockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, time.time(), "Genesis Block - Document Verification System", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_document(self, doc_name, file_bytes):
        doc_hash = hashlib.sha256(file_bytes).hexdigest()
        block_data = {
            "document_name": doc_name,
            "document_hash": doc_hash,
            "status": "Verified"
        }
        new_block = Block(
            index=len(self.chain),
            timestamp=time.time(),
            data=block_data,
            previous_hash=self.get_latest_block().hash
        )
        self.chain.append(new_block)
        return doc_hash

    def verify_document(self, file_bytes):
        target_hash = hashlib.sha256(file_bytes).hexdigest()
        for block in self.chain[1:]:
            if block.data.get("document_hash") == target_hash:
```

```

        return True, block.data, block.timestamp
    return False, None, None

if __name__ == "__main__":
    sys_blockchain = DocumentBlockchain()
    print("=====")
    print("    REAL FILE BLOCKCHAIN VERIFIER SYSTEM    ")
    print("=====")
    print("Type 'exit' at any prompt to quit the program.\n")
    while True:
        print("Choose an option:")
        print("1. REGISTER a real file to the blockchain")
        print("2. VERIFY an existing file's authenticity")
        choice = input("Enter choice (1 or 2): ").strip()
        if choice.lower() == 'exit':
            break
        if choice in ["1", "2"]:
            file_path = input("Drag & drop your file here or paste its path: ").strip()

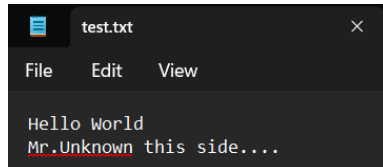
            file_path = file_path.strip("\\")
            if not os.path.exists(file_path):
                print(" ERROR: File not found! Please check the file path.\n")
                continue
            try:
                with open(file_path, "rb") as f:
                    file_bytes = f.read()
                file_name = os.path.basename(file_path)

                if choice == "1":
                    doc_hash = sys_blockchain.add_document(file_name, file_bytes)
                    print(f"\nSUCCESS: File anchored to blockchain ledger!")
                    print(f"Filename: {file_name}")
                    print(f"SHA-256 Fingerprint: {doc_hash}\n")
                    print("-" * 50)
                elif choice == "2":
                    is_valid, meta, timestamp = sys_blockchain.verify_document(file_bytes)
                    print("\n--- VERIFICATION RESULT ---")
                    if is_valid:
                        print("STATUS: VALID AND UNTAMPERED!")
                        print(f"Original Filename: {meta['document_name']}")
                        print(f"Registered On   : {time.ctime(timestamp)}")
                    else:
                        print("STATUS: INVALID / ALTERED!")
                        print("Reason: File signature does not match any block on the ledger.")
                    print("-" * 50 + "\n")

            except Exception as e:
                print(f" Error processing file: {e}\n")
            else:
                print("Invalid option. Please type 1 or 2.\n")

```

```
C:\BCTPRAC>python document_verfier_file.py
=====
REAL FILE BLOCKCHAIN VERIFIER SYSTEM
=====
Type 'exit' at any prompt to quit the program.
```



```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): 1
Drag & drop your file here or paste its path: C:\BCTPRAC\test.txt

SUCCESS: File anchored to blockchain ledger!
Filename: test.txt
SHA-256 Fingerprint: fab9b97ec69069e9346edd14b1549a1fe51d1680c016b0ad2f1769ff41a8a19b
```

```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): 1
Drag & drop your file here or paste its path: C:\BCTPRAC\bct.txt
ERROR: File not found! Please check the file path.
```

```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): 2
Drag & drop your file here or paste its path: C:\BCTPRAC\test.txt

--- VERIFICATION RESULT ---
STATUS: VALID AND UNTAMPERED!
Original Filename: test.txt
Registered On      : Sat Aug 15 00:09:59 2026
-----
```

```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): 2
Drag & drop your file here or paste its path: C:\BCTPRAC\bct.txt
ERROR: File not found! Please check the file path.
```

```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): 4
Invalid option. Please type 1 or 2.
```

```
Choose an option:
1. REGISTER a real file to the blockchain
2. VERIFY an existing file's authenticity
Enter choice (1 or 2): exit
```